



SISTEMAS OPERATIVOS - TEORÍA
13 de Junio de 2023

Nombre _____ DNI _____
Apellidos _____ Grupo _____

Cuestión 1. (2 puntos) Un sistema de ficheros basado en i-nodos y mapa de bits contiene la siguiente información:

Mapa de bits: 1 0 0 1 0 1 1 1 0 0 0 0 1 0 0 1 0 0 0
 i-nodo 2 i-nodo 3 i-nodo 4 i-nodo 5 i-nodo 9

Tamaño	1	Tamaño	2	Tamaño	1	Tamaño		Tamaño	1
#Enlaces	NA	#Enlaces	1	#Enlaces	1	#Enlaces	NA	#Enlaces	NA
Tipo F/D	D	Tipo F/D	F	Tipo F/D	F	Tipo F/D	D	Tipo F/D	D
Permisos	rwX	Permisos	rw-	Permisos	rw-	Permisos	r-x	Permisos	rwX
Directo	3	Directo	6	Directo	12	Directo	0	Directo	5
Indirecto	Null	Indirecto	7	Indirecto	Null	Indirecto	Null	Indirecto	Null

Bloque 0 Bloque 3 Bloque 5 Bloque 6 Bloque 7 Bloque 12 Bloque 15

.	5	.	2	.	9	aaaa	15	bbbb	cccc
..	2	..	2	..	5	aaaa	...	bbbb	cccc
C	9	A	3			aaaa	...	bbbb	cccc
D	4	B	5			aaaa	...	bbbb	cccc

Cada bloque físico ocupa 16 bytes. Una dirección de disco (puntero a bloque) ocupa 1 byte. Considera el siguiente código y responde a las preguntas:

```
int main( void ) {  
    char buf[16] = {[0 ... 15] = 'x'};  
    int fd1,fd2;  
    fd1 = open("/A", O_RDWR);  
    fd2 = open("/A", O_RDWR);  
    lseek(fd1,16,SEEK_CUR);  
    read(fd2,buf,8);  
    read(fd1,buf,16);  
    write(fd1,buf,8);  
    close(fd1);  
    close(fd2);  
}
```

- (0.5 puntos)** Dibuja el árbol de ficheros y directorios inicial, indicando el contenido de los ficheros regulares.
- (0.25 puntos)** ¿Cuál será el contenido del array buf tras la primera llamada a read()?
- (0.25 puntos)** ¿Cuál será el contenido del array buf tras la segunda llamada a read()?
- (1 punto)** Indica el contenido de todas las estructuras modificadas (inodos, bloques de datos y/o entradas del mapa de bits) tras ejecutar el código completo

Cuestión 2. (2 puntos) Considere un sistema monoprocesador con una política de planificación de procesos de 2 niveles de prioridad expropiativa sin realimentación. El nivel más prioritario (L1) usa una política FCFS. El menos prioritario (L2), una política de planificación expropiativa circular (round robin) con cuanto de tiempo 2.

Los procesos de cada nivel tienen los siguientes tiempos de llegada y patrones de ejecución:

Proceso	Prioridad	Tiempo de llegada	CPU	E/S	CPU
P1	L1	2	3	9	3
P2	L1	4	2	9	2
P3	L2	1	4	3	4
P4	L2	3	2	6	2
P5	L2	4	3	3	3

a) **(1,5 puntos)** Usando el diagrama de tiempos de la figura, muestre qué proceso está ejecutándose (utilice 'X1' o 'X2' para marcar el proceso en ejecución desde el nivel L1 ó L2, respectivamente, y 'O' para marcar el proceso bloqueado por E/S) y qué procesos hay en las colas de cada nivel durante las 17 primeras unidades de tiempo.

Nota: considere que si el tiempo de llegada de un proceso es 2, eso significa que puede ser planificado en el ciclo 2 y si una ráfaga de CPU o E/S dura 2, quiere decir que termina un poco antes del segundo tick de la ráfaga.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
P1																	
P2																	
P3																	
P4																	
P5																	
Cola L1																	
Cola L2																	

b) **(0,5 puntos)** Indique de forma justificada cuáles de las transiciones entre los estados de un proceso no se pueden producir en la política de planificación FCFS.

Cuestión 4. (2 puntos) En un sistema Linux se ejecuta el siguiente código:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main(void) {
    int fd1, fd2, pos, i, status;
    char c, buffer[3];
    pid_t pid, pt_id[5];

    fd1 = open("output.txt", O_CREAT | O_TRUNC | O_RDWR, S_IRUSR | S_IWUSR);
    write(fd1, "00", 2);
    pt_id[0] = getpid();
    pos = lseek(fd1, 0, SEEK_CUR);

    for (i = 1; i < 5; i++) {
        pid = fork();
        if (pid == 0) {
            fd2 = open("output.txt", O_WRONLY);
            sprintf(buffer, "%d", pos*11);
            lseek(fd2, pos, SEEK_SET);
            write(fd2, buffer, 2);
            close(fd2);
            exit(EXIT_SUCCESS);
        }
        pt_id[i] = pid;
        waitpid(pt_id[i], &status, 0);
        pos = lseek(fd1, 2, SEEK_CUR);
    }
    close(fd1);
    return 0;
}
```

a) (0.5 puntos) Explique de forma razonada cuántos procesos se crean, sin contar el proceso original, indicando cuántos ejecutan de forma concurrente y si se puede o no determinar el orden de ejecución de los mismos.

b) (1 punto) ¿Cuál es el contenido del fichero al finalizar la ejecución?

c) (0.5 puntos) ¿Qué información se guarda en el array pt_id?

Cuestión 5 (2 puntos) Un sistema utiliza gestión de memoria virtual con paginación bajo demanda con páginas de 4KiB, direcciones de memoria de 64 bits y memoria principal de 16GiB. En dicho sistema se desea ejecutar el siguiente programa:

<pre> #include <stdio.h> #include <pthread.h> /* resto de includes */ ... #define N 2048 int A[N] = {0}; int *B; struct vlimits { int start; int end; }; void *copyv(void *arg){ struct vlimits* par = (struct vlimits*) arg; int i; for (i = par->start; i <= par->end && i < N; i++){ B[i] = A[i]; } return NULL; } int main(int argc, char *argv[]) { int opt, i, offset = 0, nr_threads = 2 share; struct vlimits *targs = NULL; pthread_t *descr; // Punto A while ((opt = getopt(argc, argv, "n:h")) != -1){ switch (opt) { case 'h': fprintf(stderr, "Usage: %s [-h -n nthreads]\n", argv[0]); exit(0); case 'n': nr_threads = atoi(optarg); if (nr_threads < 0 nr_threads > 16) exit(1); break; } } } </pre>	<pre> for (i = 0; i < N; i++) A[i] = i + 1; // Punto B B = malloc(N * sizeof(int)); targs = malloc(nr_threads * sizeof(struct vlimits)); descr = malloc(nr_threads * sizeof(pthread_t)); offset = 0; share = (N + (nr_threads - 1)) / nr_threads; for (i = 0; i < nr_threads; i++){ targs[i].start = offset; targs[i].end = offset + share - 1; offset = targs[i].end + 1; } // Punto C for (i = 0; i < nr_threads; i++) pthread_create(&descr[i], NULL, copyv, (void *)&targs[i]); // Punto D for (i = 0; i < nr_threads; i++) pthread_join(descr[i], NULL); for (i=0; i<N; i++) printf("B[%i]=%i\n",i,B[i]); free(B); free(targs); free(descr); // Punto E return 0; } </pre>
---	--

Considerando que:

- Las llamadas al sistema y el resto de funciones de librería que se incluyen en el programa nunca fallan.
- Todas las funciones externas se incluyen en una única biblioteca cuyo texto ocupa 3,7 KiB, y su sección de datos (todos inicializados) ocupa 2,9 KB. Dicha biblioteca se enlazó de forma dinámica al crear el ejecutable.
- El texto (código) del programa tras la compilación ocupa 2KiB. Dicha región de texto incluye el código para la carga dinámica de bibliotecas, y se ha de suponer que la región de datos inicializados de la biblioteca se modifica durante su carga, ya que ésta incluye un código de inicialización.
- El contenido inicial de la pila al lanzar a ejecución el programa ocupa 256 bytes.
- Un entero en C ocupa 4 bytes, y el tipo de datos pthread_t ocupa 8 bytes.

- a) **(1 punto)** Para cada variable, macro o expresión de C listada a continuación, indique si el símbolo correspondiente ocupa espacio en el ejecutable y en caso afirmativo cuál es la sección del mismo a la que pertenece. Especifique también en qué región del mapa de memoria del proceso se encuentra el símbolo asociado, así como el número de bytes que este ocupa en memoria (si aplica).

Variable /Macro / Expresión C	Presente en ejecutable (Sí/No)	Sección del ejecutable	Región del mapa de memoria	Tamaño en bytes del símbolo asociado
N				
A				
B				
targs				
targs[0]				
optarg				

- b) **(1 punto)** Identifique qué regiones lógicas (y su tamaño en páginas) constituyen la imagen de memoria del proceso multihilo en los puntos marcados en el código como A, B, C, D y E,.